



LOST CHILDREN OF PERU · DOCUMENTACIÓN TÉCNICA

Módulo RRHH por dentro

Cómo se instala, cómo se despliega, qué cuidados necesita y qué decisiones se tomaron por el camino. Para quien tenga que mantenerlo.

VERSIÓN

2 de setiembre de 2026

REPOSITORIO

elizzaync/erp-lost-children

PRUEBAS

94 de 94

De qué está hecho

Deliberadamente poco. Cuantas menos piezas, menos cosas que administrar en una ONG que no tiene un equipo de sistemas.

PIEZA	QUÉ ES
Python 3.11	El lenguaje del servidor.
Flask	Sirve la API y la propia interfaz, en el mismo origen. Sin CORS que configurar.
SQLite	La base entera es un archivo . No hay servidor de base de datos que instalar, arrancar ni respaldar aparte. Viene con Python.
requests	Cliente HTTP hacia yunatt.com. Su urllib3 es el que permite forzar TLS 1.3, que el handshake con yunatt exige en Windows.
gunicorn	Solo en el contenedor. En la oficina se usa el servidor de Flask.

Eso es todo: **dos dependencias** para funcionar y una tercera para el contenedor. La interfaz no usa ningún framework.

Son unas 23.500 líneas entre servidor e interfaz, y **29 tablas**.

Las versiones no están fijadas a propósito

Se piden mínimos (flask $\geq$ 3.0), no versiones exactas. Esta máquina comparte el Python del sistema con el ERP anterior; fijar una versión forzaría una bajada que afectaría también a ese sistema mientras siga en producción.

Correrlo en la oficina

Es lo que se usa hoy, y lo que se usará en la presentación al cliente.

```
# Desde la carpeta del proyecto
py backend\servidor.py

# Y en el navegador
http://127.0.0.1:7801/
```

Se arranca por `servidor.py` y no por `app.py` por una razón concreta: si el 7801 ya está ocupado —un arranque duplicado, o una consola que se cerró dejando el proceso vivo— Flask fallaría o el sistema operativo entregaría la conexión al proceso viejo. El resultado sería la misma dirección mostrando código antiguo o una base distinta, y pareciendo que «el login no funciona». `servidor.py` libera el puerto antes de arrancar, en vez de saltar a otro.

Los puertos están repartidos

7801

El servidor de la oficina

7802

El banco de las pruebas

7803Login en modo
convivencia**7804**

Login en modo estricto

Levantar cualquier otra cosa en el 7802, 7803 o 7804 rompe suites sin que se note de dónde viene el fallo: la prueba habla con el servidor equivocado y acusa al sistema de algo que no hizo.

La interfaz es un archivo generado

Nunca se edita el `.dc.html` a mano

`ERP RRHH - Lost Children Peru.dc.html` se reconstruye en cada arranque desde `interfaz/`. Cualquier cambio hecho directamente sobre él se pierde al reiniciar.

CARPETA	QUÉ CONTIENE
<code>interfaz/base.html</code>	El armazón: barra lateral, cabecera, pie.
<code>interfaz/pantallas/</code>	Una pieza por pantalla.
<code>interfaz/dialogos/</code>	Los cuadros de diálogo.
<code>interfaz/logica/</code>	El JavaScript, repartido por módulo.
<code>construir_interfaz.py</code>	Los junta y escribe el <code>.dc.html</code> .

El ensamblado ocurre al importar `app.py`, así que editar una pieza y recargar el navegador basta. Si falta una pieza, el arranque revienta a propósito: servir la versión vieja en silencio sería peor que no arrancar.

Antes de dar por bueno un cambio de interfaz

```
py verifica_ids.py
```

Comprueba tres cosas que no dan error por sí solas y rompen pantallas enteras: que todo `{{ nombre }}` del marcado exista de verdad, que los `<div>` cierren dentro de cada bloque, y —desde el 31/08/2026— que no se haya colado un **carácter de control** ni un **comentario sin cerrar**.

Las dos últimas comprobaciones nacieron de fallos reales

Un `0x08` invisible dentro de una expresión regular hacía que una función devolviera vacío para todas las fechas, y el panel anunciaba que ninguna ficha tenía fecha de ingreso teniéndolas todas.

Y un `/*` sin su `*/` se comió media pantalla en silencio: la lista de enrolamiento aparecía vacía, a cientos de líneas del destrozo.

Desplegar en Coolify

Coolify vigila la rama `main`. Subir a main **es** desplegar.

- 1 Correr la regresión completa. Si no está verde, no se sube.
- 2 `git push origin main`.
- 3 Coolify reconstruye la imagen y levanta el contenedor.
- 4 Comprobar que `/api/health` responde antes de dar la dirección a nadie.

El volumen de datos no puede faltar

La base vive en `/app/data`. Si ese directorio no es un volumen persistente, **cada redesplicue empieza con la base vacía** y se lleva por delante los documentos y contratos que haya subido el equipo.

Un solo trabajador, y no es negociable

```
gunicorn --chdir backend wsgi:app -b 0.0.0.0:7801 -w 1 --threads 8
```

El seguimiento de un enrolamiento vive en un diccionario en la memoria del proceso. Con dos trabajadores, una captura empezada en uno no existiría para el otro y la pantalla preguntaría por un enrolamiento «que no existe».

De ahí sale también por qué las pantallas se actualizan preguntando cada doce segundos en vez de por websockets: con ocho hilos, ocho pestañas abiertas dejarían al servidor sin atender nada, ni siquiera marcar. El día que el enrolamiento salga de la memoria del proceso, se puede cambiar sin tocar ninguna pantalla.

Las variables

En la oficina van en `backend/.env`, que no sube al repositorio. En el servidor, en el panel de Coolify.

VARIABLE	PARA QUÉ
YUNATT_EMAIL YUNATT_PASSWORD	La cuenta de yunatt.com. Sin ellas el sistema arranca igual, en modo «sin credenciales», y lo dice en pantalla: no se puede enrolar ni traer marcas, el resto funciona.
YUNATT_DEVICE_ID	El identificador interno del Timmy en yunatt. No es su número de serie.
YUNATT_DEPT_NAME	El departamento donde cae todo lo que enrole este sistema. Sirve para distinguir en yunatt qué entró por aquí y qué por el ERP anterior.
LOGIN_ESTRICTO	1 por defecto. En 0 abre la puerta «entrar sin cuenta», que solo tenía sentido antes de repartir las cuentas.
COOKIE_SECURE	1 por defecto. Marca la cookie de sesión para que el navegador solo la mande por https.
LUGARES_ACTIVO	1 por defecto. En 0 deja de preguntar el nombre de los sitios a OpenStreetMap.
YUNATT_BASE	Dirección de yunatt. Sin ponerla, <code>https://global.yunatt.com</code> . Hoy hace falta: <code>http://www.yunatt.com:82</code> .
SINCRONIZAR_CADA	45 por defecto. Cada cuántos segundos se traen las marcas del terminal. En 0 se apaga.
PUERTO	7801.

Por qué no entra nadie sin https

Es a propósito, y falla cerrado

`COOKIE_SECURE` viene en 1: el navegador solo manda la cookie de sesión por una conexión segura. Sin certificado no hay sesión, y por tanto nadie entra al sistema desplegado.

Es la conducta correcta. Una contraseña viajando en claro por la red de la casa es exactamente lo que no puede pasar. Pero conviene saberlo antes de enseñar la dirección a alguien y descubrir que no deja entrar.

Mientras no haya dominio

Para enseñar el sistema desde un celular hay un túnel que da una dirección `https` temporal contra la computadora de la oficina:

```
herramientas\tunel-rrhh.bat
```

- La dirección **cambia cada vez**. No sirve guardarla.
- La computadora tiene que quedarse encendida con el servidor arriba.
- Sirve para **demostrar** que funciona, no para trabajar con datos reales de niños: el tráfico pasa por un tercero.
- La primera vez descarga **cloudflared** —52 MB, un solo .exe sin instalador ni cuenta— y lo deja junto al .bat. Queda fuera del repositorio: es binario de terceros, no código del proyecto.

Comprueba el servidor antes de levantar el túnel

Si el 7801 está apagado, el túnel se levanta igual y entrega una dirección que contesta **502**. Parece que falla el túnel cuando lo que falta es arrancar el servidor. Por eso el .bat pregunta primero por `/api/health` y, si no responde, se detiene y dice qué hay que arrancar en vez de dar una dirección rota.

El día que haya dominio con certificado, se configura en el panel de Coolify y ya. No hace falta tocar código ni reconstruir la imagen.

El terminal y yunatt

El Timmy TM-AI03F no se habla directamente: se habla con yunatt.com, que es quien lo controla.

La cuenta ya no se comparte con el ERP anterior

Hasta el 01/09/2026 los dos sistemas usaban la misma cuenta y se tiraban la sesión el uno al otro. Ese día la plataforma que usaban se cayó, la cuenta antigua dejó de admitir el acceso, y se migró a una cuenta propia. Ahora nada se pisa.

La dirección cambió, y no era un apaño

Se venía usando <https://global.yunatt.com>. El 01/09/2026 dejó de responder: su servidor contesta, pero no hay nada escuchando en el 443. La dirección buena es <http://www.yunatt.com:82> —lo dice el manual del propio fabricante, textualmente— y son **servidores distintos**: una cuenta creada en uno no existe en el otro.

Ese puerto va sin cifrar

En el 82 no ofrecen HTTPS, así que la contraseña viaja en claro hasta ellos. Es cómo lo tienen montado, no una elección de este sistema. La dirección se lee de `YUNATT_BASE` en el `.env`: el día que vuelva la de siempre, se recupera el cifrado **borrando esa línea**, sin tocar código ni desplegar.

Tres cosas que esta plataforma hace distinto

Descubiertas midiendo contra el sistema real, no leyendo documentación:

QUÉ	CÓMO SE RESOLVIÓ
No existe queryStaff	La ruta que daba la lista de quién está en el terminal responde 404. Ahora se deduce de la ficha en la nube.
faceNum miente	Se enroló a una persona real —el Timmy la reconocía— y su contador de rostros siguió en 0, incluso tras pedir <code>/staff/getFromMachine</code> . La señal fiable es la foto : el terminal la sube al capturar, y es además la que acaba en la ficha.
No se puede borrar personal	<code>/staff/batchRemove</code> y cinco variantes más dan 404. Los registros huérfanos se quitan a mano desde su panel. No estorban: quien no está en el terminal no puede fichar.

La zona horaria de la cuenta: la trampa que más cuesta ver

Las cuentas nuevas vienen en **Asia/Singapore**, trece horas por delante de Perú. El terminal registra bien, pero la plataforma fecha un fichaje de las 15:00 como las 04:00 **del día siguiente**. El sistema busca las marcas de hoy y no encuentra nada, así que parece que el Timmy no manda.

Se arregla en su panel: *Company Information* → *timeZone* → **America/Lima**. Y desde hoy el sistema además **descarta** cualquier marca fechada por delante del reloj —nadie ficha en el futuro— y lo avisa en el registro, para que el síntoma señale a su causa.

El rango reservado

Este sistema solo puede tocar identificadores **desde el 9000**. Nació para no pisar al ERP anterior cuando compartían cuenta; se mantiene porque sigue siendo la barrera que impide que una prueba borre a alguien real.

Si el proveedor se cae, no se pierde asistencia

Medido el 03/09/2026: al terminal se le quitó la red, alguien fichó, se verificó que esa marca **no** había llegado a la nube, y al devolverle la señal apareció tres minutos después. El equipo guarda dentro lo que no puede enviar.

Lo que sí conviene saber: **durante la caída, Asistencia no muestra lo que ha pasado sino lo que ha llegado.** Puede estar vacía mientras la gente ficha con normalidad.

Las marcas entran solas

Un hilo de fondo (`backend/sincronizador.py`) pregunta a yunatt cada **45 segundos** y guarda lo nuevo. Antes las marcas solo entraban al pulsar «sincronizar»: alguien fichaba y el sistema no se enteraba hasta que una persona lo pedía.

- Un fallo del proveedor **nunca** sale de ese hilo: se apunta y se sigue. Su plataforma va irregular y no puede tumbar el servidor.
- Tras fallar espera cada vez más, hasta 5 minutos, en vez de insistir contra un servidor que ya dijo que no. Al volver, ritmo normal enseguida.
- Arranca desde `servidor.py` y `wsgi.py` , **no al importar**: las suites importan `app` decenas de veces y no deben abrir hilos hablando con yunatt de verdad.
- Se apaga con `SINCRONIZAR_CADA=0` .

Las pantallas ya vigilaban la base cada pocos segundos, así que en cuanto una marca entra aparece sola. Ese sondeo bajó de 12 a **5 segundos**: la consulta no devuelve datos, solo un sello que cambia, y con la pestaña de fondo ni se pregunta.

Quitar a alguien ya no borra su asistencia

Era una pérdida de datos silenciosa

`marcas.staff_number` apunta a `identidades` con `ON DELETE CASCADE`, así que retirar a una persona del terminal se llevaba por delante **todo su historial de asistencia**. Ocurrió de verdad el 01/09/2026 con dos personas.

Ahora la fila no se borra: se marca **retirada** y se le ponen los biométricos a cero. Las marcas se conservan —son un hecho, no dejan de haber ocurrido—, la persona reaparece como enrolable, y al volver a enrolarla **reutiliza su mismo número**, así que su historial sigue siendo suyo.

Cuando un enrolamiento se queda a medias

El seguimiento vive en la memoria del proceso: si la pantalla se cierra o se reinicia el servidor, nadie vuelve a preguntarle al equipo y la ficha se quedaría en «esperando» para siempre. Al entrar en Gestión Biométrica, el sistema le pregunta al terminal por todas las identidades a medias y las pone al día.

El rostro del celular

Marcar desde el teléfono compara la cara con una referencia. Todo el reconocimiento ocurre **en el navegador**.

QUÉ	DETALLE
Librería	face-api 1.7.15, licencia MIT, alojada en web/rostro/ — 7,4 MB. No se descarga de internet: si el servidor no tiene salida, sigue funcionando.
Qué se guarda	128 números por persona. No la fotografía de referencia.
Umbral	0,6 de distancia. Medido: la misma cara consigo misma da entre 0,03 y 0,14; caras distintas, entre 0,64 y 0,89.
Foto de la marca	Esa sí se guarda, en data/marcas/ . Es la prueba de quién fichó ese día.

Cambiar de modelo obliga a re-enrolar a todo el mundo

Los 128 números solo tienen sentido dentro del modelo que los calculó. Si algún día se actualiza face-api, las referencias guardadas dejan de servir y hay que volver a registrar el rostro de cada persona.

El nombre de los sitios

Unas coordenadas no dicen dónde está nadie. El nombre hay que preguntárselo a alguien que tenga el mapa.

Se usa **Nominatim**, el servicio de OpenStreetMap: gratuito, de una fundación sin ánimo de lucro, sin registro ni clave.

Cada consulta le dice a un tercero dónde estuvo alguien

No va el nombre de la persona —solo las coordenadas— pero sí el lugar y el momento. Por eso solo se consulta **al guardar** una marca, nunca al pintar una pantalla: si se hiciera al mirar, abrir el registro de asistencia mandaría fuera la ubicación de todo el equipo cada vez.

- El nombre **se guarda con la marca**. Si el servicio desaparece, lo ya registrado sigue ahí.
- Las coordenadas se redondean y la respuesta se recuerda: diez personas fichando en la misma puerta son una consulta, no diez.
- Si no contesta en tres segundos, la marca entra igual, sin nombre.
- Se apaga con `LUGARES_ACTIVO=0`.

La base y los respaldos

Toda la información del sistema está en un solo archivo: `data/rrhh.db`.

Eso hace el respaldo trivial —copiar ese archivo— y lo hace también el punto único de fallo. Junto a él viven las carpetas de adjuntos:

CARPETA	QUÉ GUARDA
<code>data/fotos/</code>	Las fotos de las fichas, incluidas las de menores.
<code>data/marcas/</code>	Una foto por fichaje desde el celular.
<code>data/firmas/</code>	Las firmas dibujadas.
<code>data/archivos/</code>	Documentos y contratos que sube el equipo.
<code>data/credenciales/</code>	La llave de Google. Nunca sube al repositorio.

Un respaldo de la base sin sus carpetas está incompleto

La base guarda el **nombre** de cada archivo, no el archivo. Restaurar solo el `.db` deja todas las fichas apuntando a fotos y documentos que no existen.

Migraciones

El esquema se actualiza solo al arrancar: `db.iniciar()` crea lo que falte y añade las columnas nuevas, sin tocar los datos. Es idempotente, así que reiniciar no molesta.

Una tabla no puede declararse dos veces

La lista de columnas nuevas es un diccionario. Si una tabla aparece dos veces, Python se queda con la última y **descarta la primera entera**: sus columnas no se crean nunca y el síntoma aparece mucho después, como un campo que «no se guarda». Pasó de verdad. Hay una prueba que lo impide.

Seguridad

- **Las contraseñas** no se guardan: se guarda su hash.
- **Los tokens de sesión** tampoco. En la tabla queda su huella SHA-256, así que quien lea la base no puede suplantar a nadie.
- **Toda escritura exige el token CSRF**, que viaja en el cuerpo del login y se devuelve en una cabecera.
- **Los permisos son por rol**, y de tres niveles por módulo: sin acceso, vista o edición. Quien no tiene vista sobre Beneficiarios no ve esa parte del menú siquiera.
- **Quien resuelve queda anotado**: la firma de un permiso sale de la sesión de quien lo aprobó, nunca de lo que venga en la petición.
- **La sesión caduca** a los 45 minutos sin uso.

El repositorio es público

Quedan fuera, por `.gitignore` : la base y sus respaldos, las fotos de fichas y de fichajes, las firmas, los documentos del personal, la llave de Google y `backend/.env` .

Antes de cada subida conviene comprobarlo. En una ocasión estuvieron a punto de irse 80 fotografías de caras y 42 firmas.

Las pruebas

94 suites, todas en verde. Unas hablan con la API; otras abren un navegador de verdad y pulsan botones.

```
# Todas, unos 30 minutos
cd pruebas
py correr_todo.py

# Una sola
py correr_una.py prueba_marcar_foto

# Y antes de dar por bueno cualquier cambio de interfaz
py verifica_ids.py
```

Viven en `pruebas/`, dentro del repositorio. Deducen la raíz del proyecto de su propia ubicación, así que funcionan en cualquier clon sin configurar nada.

`pruebas/archivo/` guarda catorce suites que **no** están en la lista del corredor: restos de trabajo, versiones anteriores de suites que sí corren, y una rota que lee un archivo inexistente. No protegen nada. Antes de recuperar cualquiera hay que correrla contra el sistema de hoy.

Nunca las dos cosas a la vez

La corrida completa y una suite suelta usan el mismo puerto 7802.

Lanzar una mientras corre la otra produce decenas de fallos falsos —«conexión denegada»— que no tienen nada que ver con el sistema.

Lo mismo vale para reconstruir la interfaz a mitad de una corrida: las suites de navegador cargan el archivo que se está reescribiendo.

Nunca tocan la base real

Cada corrida hace una **copia** de `data/rrhh.db`, levanta su propio servidor contra ella y la borra al terminar. Hay una suite dedicada — `prueba_aislamiento` — que lo comprueba matando el proceso en seco y verificando que la base real quedó exactamente como estaba.

Datos de ejemplo

El sistema trae hoy un juego de fichas inventadas para poder enseñarlo funcionando.

```
# Las personas: equipo, niños, responsables, vínculos y un mes de marcas
py backend\sembrar_ejemplo.py      # enseña el plan
py backend\sembrar_ejemplo.py --ejecutar

# Los expedientes: documentos, sueldos, sesiones, incidencias...
py backend\sembrar_expedientes.py --ejecutar

# Y para retirarlo todo
py backend\sembrar_ejemplo.py --borrar
py backend\sembrar_expedientes.py --borrar
```

- Los documentos van en el bloque **90.000.000–90.999.999**, que RENIEC no emite: ninguna ficha inventada puede coincidir con el DNI de una persona real.
- El borrado va **por identificador**, no por nombre: si alguien corrige un nombre desde la pantalla, la ficha sigue localizable.
- `sembrar_expedientes` comprueba ese bloque antes de escribir. En una base con datos reales no hace nada.

Retirarlas antes de empezar de verdad

Se quedan para la presentación al cliente —sin datos, las pantallas salen vacías y los gráficos no se ven—. El día que el equipo empiece a registrar información real, tienen que salir.

Lo que queda pendiente

Escrito aquí para que no dependa de que alguien se acuerde.

PENDIENTE	DETALLE
El dominio	Aplazado hasta después de la entrevista con el cliente. Sin él, el sistema desplegado no deja entrar a nadie.
Enrolar al equipo real	Hoy solo están enroladas las fichas de ejemplo.
Re-enrolar borra marcas	Cambiar el enrolamiento de alguien le cambia el número y sus marcas anteriores se van con el viejo. Sin resolver.
Cuentas de prueba	<code>hari</code> y <code>harito</code> se retiran junto con los datos de ejemplo.
Ley 29733	El aviso de tratamiento de datos cubre al adulto que lo firma. Cuando entren datos de niños por el formulario, hay que reescribirlo.
Primer respaldo fuera	Subir una copia de <code>data/</code> a un sitio distinto del servidor.